PAGES
- 01 · Cover
- 02 · System Architecture
- 03 · Auth & Multi-Tenancy
- 04 · ER Overview Map

SCHEMA FRAMES
- 05 · Platform & Identity
- 06 · Academics
- 07 · Students & Admissions
- 08 · Teachers
- 09 · Attendance & Biometric
- 10 · Exams & Promotion
- 11 · Fees & Billing
- 12 · Classroom (Rooms)
- 13 · Ratings & Notifications

FLOWS
- 14 · Domain Data Flows
- 15 · API Surface Map

PROTOTYPE · v1.0 · 2026-08-11

Multi-Vendor School Management System

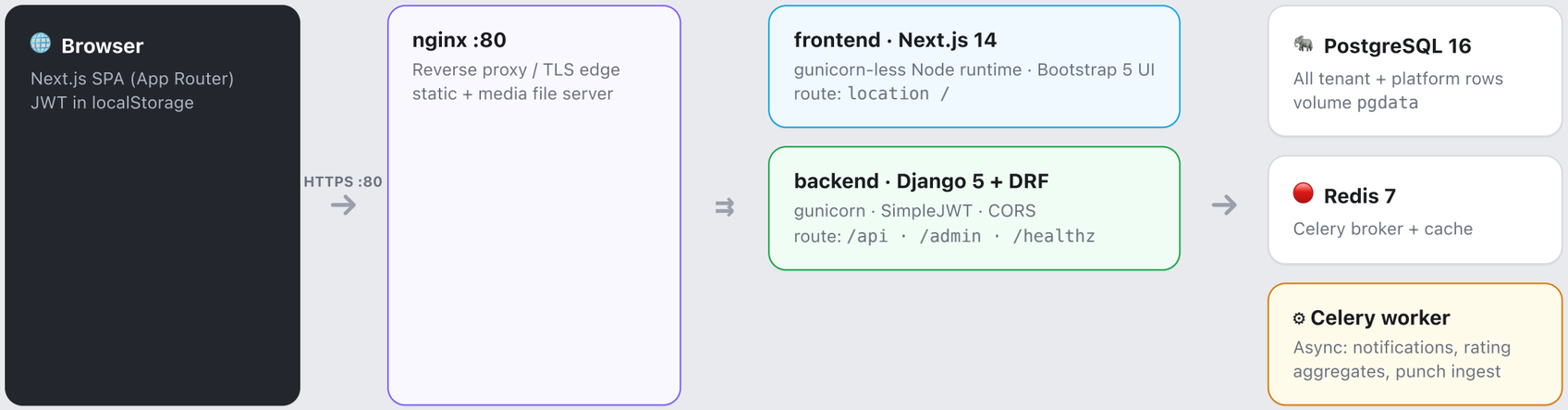
Data-flow diagrams & database table outline — a Figma-style walkthrough of the decoupled Django REST + Next.js multi-tenant platform.

12	40	1	40+
DJANGO APPS	DB TABLES	TENANT ROOT (SCHOOL)	REST RESOURCES

Stack: Django 5 · DRF · SimpleJWT · PostgreSQL 16 · Redis · Celery · Next.js 14 · nginx · Docker • Subdomain-per-tenant SaaS

How a request moves through the stack

Every browser hit lands on **nginx**, which forks traffic: `/api · /admin · /static · /media` to the Django backend, everything else to the Next.js frontend. The backend is the only tier that touches Postgres, Redis, and Celery.



NGINX ROUTING TABLE

`/static/`	→ shared static volume
`/media/`	→ uploads (photos, PDFs, certs)
`^/(api|admin|healthz)/`	→ upstream **backend**
`/`	→ upstream **frontend**

DOCKER-COMPOSE SERVICES

db · postgres:16-alpine · redis · redis:7-alpine · backend · ./backend (gunicorn) · frontend · ./frontend (Next.js) · nginx · 1.27-alpine (:80 exposed)

Backend + frontend are **expose**-only; nginx is the sole published port. `SEED_DEMO=1` seeds demo data on first boot.

JWT login and tenant isolation

Two roles only: **Super Admin** (school = null) and **School Admin** (pinned to one school). Tenant scope is resolved server-side on every request — the subdomain never overrides a user's own `school_id` (defense in depth against spoofed hosts / shared cookies).

LOGIN & TOKEN FLOW

1. **POST /api/auth/login/** — `{username,password}` → access + refresh tokens + user object.
2. Frontend stores tokens in `localStorage` (`sms_access`, `sms_refresh`). Every call attaches `Authorization: Bearer`.
3. On **401**, api.js transparently calls **/api/auth/refresh/** once, then replays the original request.
4. Super Admin "enters" a tenant → id sent as `X-Active-School` header (honored only for super_admin; stored in session as `active_school_id`).

TENANTMIDDLEWARE RESOLUTION ORDER

① Host → slug
`greenvalley.platform.local` → School(slug="greenvalley"). Reserved: www/app/admin/api/static/media. Custom domain matched first (Premium).

↓

② Auth override (wins)
Regular user → **always** pinned to `user.school`. Super Admin → session `active_school_id` or unscoped. Subdomain ignored for logged-in users.

↓

③ Bind scope
Sets `request.school` + thread-local. **SchoolScopedManager.objects** auto-filters every query by it; `all_objects` bypasses for platform code.

TENANT ISOLATION GUARANTEE

Every business table inherits `SchoolScopedModel` → carries a **school FK** and the tenant-aware default manager. Because scope comes from the authenticated user (not the URL), a School-A user can never read School-B rows even via B's subdomain. `School`, `User`, `SubscriptionPlan`, and `ContactMessage` are the only non-scoped / platform-level tables.

School is the tenant root; 10 domains hang off it

A radial view of how the 40 tables cluster. **School** is the hub every scoped row points to. Colored clusters = the schema frames that follow (05–13).



Tenant root — every scoped table has `school_id` Arrows = "belongs to School" Inner cross-domain FKs shown in detail frames 05–13

tenants + accounts

These four tables sit **above** tenancy. `School` is the tenant; `SubscriptionPlan` gates limits; `ContactMessage` is the sales-lead funnel; `User` is the only login table (RBAC via `role`).

School — tenants.School

id PK	bigint
name / slug UQ	char/slug
custom_domain UQ	char null
school_type	choice
plan_id FK	→ SubscriptionPlan
status	choice(7)
trial_ends_at	datetime
admissions_open / mode	bool/choice
device_brand / token	choice/char
attendance_late_after	time
primary_color / logo	char/img

SubscriptionPlan — tenants

id PK	bigint
name	char
tier UQ	choice(5)
student_limit	int null
price_monthly/yearly	decimal
features	jsonb
is_active	bool

ContactMessage — tenants

id PK	bigint
name / email / phone	char
school_name / topic	char
message	text
is_handled	bool
converted_school_id FK	→ School null

User — accounts.User

id PK	bigint
username UQ	char
password / email	char
role	super_admin·school_admin
school_id FK	→ School null
phone	char
is_staff/superuser	bool

Extends Django `AbstractUser`. `school=null` ⇒ Super Admin. Students & guardians may hold a User for portal login (1:1 / FK from students app).

Year → Class → Section → Routine

The academic hierarchy that almost every other domain references. `AcademicYear` and `SchoolClass` are roots; `Section` is the working unit students belong to; `RoutineSlot` is the weekly timetable grid cell.

AcademicYear — academics

id PK	bigint
school_id FK	→ School
name UQ*	char
start_date / end_date	date
is_current	bool

SchoolClass — academics

id PK	bigint
school_id FK	→ School
name UQ*	char
numeric_level	int (promo order)

Subject — academics

id PK	bigint
school_id FK	→ School
school_class_id FK	→ SchoolClass null
name / code UQ*	char

Section — academics

id PK	bigint
school_id FK	→ School
school_class_id FK	→ SchoolClass
academic_year_id FK	→ AcademicYear
name UQ*	A/B/C
class_teacher_id FK	→ TeacherProfile null

RoutineSlot — academics

id PK	bigint
school_id FK	→ School
section_id FK	→ Section
day_of_week / period UQ*	int
start_time / end_time	time
subject_id FK	→ Subject null
teacher_id FK	→ TeacherProfile null
room / is_break	char/bool

UQ* = composite unique scoped by `school`. RoutineSlot is unique per `(section, day, period)` — no double-booking a section's slot.

From public application to enrolled student

`AdmissionApplication` is the public intake; on approval it spawns a `Student` (+ optional portal `User`). Students carry `Guardians`, `Certificates`, and `StudentTransfer` history.

Student	students.Student
id PK	bigint
school_id FK	→ School
user_id FK	→ User 1:1 null
admission_number UQ*	char
roll_number / full_name	char
dob / gender / blood	date/choice
current_class_id FK	→ SchoolClass
current_section_id FK	→ Section
academic_year_id FK	→ AcademicYear
status	active·alumni·…
medical_notes / photo	text/img

Guardian	students
id PK	bigint
school_id FK	→ School
student_id FK	→ Student
user_id FK	→ User null (portal)
relation	father·mother·other
name / phone / nid	char
is_primary_contact	bool

Certificate	students
id PK	bigint
student_id FK	→ Student
cert_type / title	choice/char
issue_date / body	date/text

AdmissionApplication	students
id PK	bigint
school_id FK	→ School
status	pending·appr·rej
tracking_code / track_token	char (public)
full_name / dob / gender	applicant
father_* / mother_*	char
birth/transfer_certificate	file
reviewed_by_id FK	→ User null
created_student_id FK	→ Student null

StudentTransfer	students
student_id FK	→ Student
from_section / to_section FK	→ Section
transfer_date / reason	date/text

Staff records, designations, history

Teachers are **records, not logins** — only Super/School Admin have accounts. `TeacherProfile` M2M-links to Subjects & Sections; `DesignationHistory` is the promotion audit trail.

TeacherProfile	teachers
id PK	bigint
school_id FK	→ School
user_id FK	→ User 1:1 null
full_name / email / phone	char
employee_id UQ*	char
designation_id FK	→ Designation null
contract_type / status	choice
subjects M2M	↔ Subject
sections M2M	↔ Section
salary_basic	decimal

Designation	teachers
id PK	bigint
school_id FK	→ School
name UQ*	char
rank_order	int (seniority)
permissions_group	char

Configurable per school. Lower `rank_order` = more senior. Referenced by RoutineSlot, Section.class_teacher, ExamSubject.invigilator, and Rooms.

DesignationHistory	teachers
id PK	bigint
school_id FK	→ School
teacher_id FK	→ TeacherProfile
old_designation_id FK	→ Designation null
new_designation_id FK	→ Designation null
changed_at	date
changed_by_id FK	→ User null

Device punches → resolved attendance

Fingerprint devices POST raw punches to a token-authenticated webhook. `BiometricEnrollment` maps a device user-id to a Student **or** Teacher; resolved punches write `StudentAttendance` / `StaffAttendance`.

BiometricPunch — attendance

id PK	bigint
school_id FK	→ School
device_user_id	char idx
punched_at / direction	datetime
enrollment_id FK	→ BioEnrollment null
status	applied·unresolved·dup
raw	jsonb (audit)

BiometricEnrollment — attendance

device_user_id UQ*	char
student_id FK	→ Student null
teacher_id FK	→ TeacherProfile null
label	cached name

StudentAttendance — attendance

id PK	bigint
school_id FK	→ School
student_id FK	→ Student
section_id FK	→ Section
date UQ*	date
status	present·absent·late·leave
marked_by_id FK	→ User null

Unique `(student, date)` — one row per student per day.

StaffAttendance — attendance

teacher_id FK	→ TeacherProfile
date UQ*	date
check_in / check_out	time
is_present	bool

LeaveApplication — attendance

student_id FK	→ Student
from_date / to_date	date
reason / status	text/choice
reviewed_by_id FK	→ User null

Session → Paper → Result → Promotion

`ExamSession` groups papers (`ExamSubject`) per class; each student gets one `ExamResult` per paper. A `is_final` session unlocks class promotion, audited in `StudentPromotion`.

ExamSession	exams
id PK	bigint
school_id FK	→ School
academic_year_id FK	→ AcademicYear
name	char
is_published	bool
is_final	bool (promo)

ExamSubject	exams
id PK	bigint
exam_session_id FK	→ ExamSession
subject_id FK	→ Subject
school_class_id FK	→ SchoolClass
exam_date / times	date/time
invigilator_id FK	→ Teacher null
total_marks / pass_marks	int
UQ*(sess,subj,class)	

ExamResult	exams
id PK	bigint
exam_subject_id FK	→ ExamSubject
student_id FK	→ Student
marks_obtained	decimal
grade / grade_point	char/dec
is_absent	bool
entered_by_id FK	→ User null
UQ*(subj,student)	

StudentPromotion	exams
id PK	bigint
exam_session_id FK	→ ExamSession
student_id FK	→ Student
from_class / to_class FK	→ SchoolClass
percentage / rank	dec/int
promoted	bool
promoted_by_id FK	→ User null

Fee catalog → Invoice → Payment

FeeType defines categories; `FeeStructure` sets amounts (per year/class); per-student `StudentFeeDiscount` applies. `Invoice` snapshots line-items as JSON; `Payment` records settle it.

FeeType	fees
id PK	bigint
school_id FK	→ School
name UQ*	char
fee_category / frequency	choice
is_mandatory / active	bool

FeeStructure	fees
fee_type_id FK	→ FeeType
academic_year_id FK	→ AcademicYear
applicable_class_id FK	→ SchoolClass null
amount UQ*	decimal

Invoice	fees
id PK	bigint
school_id FK	→ School
student_id FK	→ Student
academic_year_id FK	→ AcademicYear null
invoice_number UQ*	char
line_items	jsonb
subtotal / discount / total	decimal
status	draft·paid·overdue…
design_template_id FK	→ InvoiceDesign null

Payment	fees
id PK	bigint
invoice_id FK	→ Invoice
amount_paid	decimal
payment_method	cash·online·bank
transaction_id / paid_at	char/dt
received_by_id FK	→ User null

StudentFeeDiscount / InvoiceDesign	fees
student_id FK	→ Student
fee_type_id FK	→ FeeType
discount_value_type / amount	flat·percent
InvoiceDesign.template_slug	classic·modern…

Room → Posts · Notes · Assignments

A `ClassRoom` = Subject × Section × Year, joinable by `room_code`. Students enroll via `RoomMembership`; teachers post announcements, notes, and assignments that students submit & get graded.

ClassRoom — rooms

id PK	bigint
school_id FK	→ School
room_code UQ*	char
subject_id FK	→ Subject
section_id FK	→ Section
academic_year_id FK	→ AcademicYear
teachers M2M	↔ TeacherProfile
created_by_id FK	→ User null

RoomMembership — rooms

room_id / student_id FK	UQ*
is_active	bool

RoomPost — rooms

id PK	bigint
room_id FK	→ ClassRoom
author_id FK	→ User null
content / attachment	text/file
post_type / is_pinned	choice/bool

RoomPostComment / Reaction

post_id FK	→ RoomPost
author/user_id FK	→ User
kind (reaction) UQ*	like·question

Note — rooms

id PK	bigint
room_id FK	→ ClassRoom
title / file / link	char/file/url
topic / chapter	char
uploaded_by_id FK	→ User null

Assignment — rooms

room_id FK	→ ClassRoom
title / instructions	char/text
due_datetime / total_marks	dt/int

AssignmentSubmission — rooms

id PK	bigint
assignment_id FK	→ Assignment
student_id FK	→ Student
submitted_files	jsonb
text_answer / is_late	text/bool
marks_obtained / feedback	int/text
graded_by_id FK	→ User null
UQ*(assign,student)	

Anonymous teacher ratings + message log

Students rate teachers per `RatingPeriod` across 6 criteria (anti-spam: one per teacher/period). Averages are denormalized into `TeacherRatingAggregate` (public only after ≥5 ratings). `Notification` logs every SMS/push/email/in-app message.

TeacherRating — ratings

id PK	bigint
school_id FK	→ School
teacher_id FK	→ TeacherProfile
student_id FK	→ Student
rating_period_id FK	→ RatingPeriod
room_id FK	→ ClassRoom null
score_* (×6)	1–5
text_comment	text (admin-only)
UQ*(teacher,stu,period)	

RatingPeriod — ratings

id PK	bigint
school_id FK	→ School
name / start / end	char/date
is_active	bool
eligible_classes M2M	↔ SchoolClass

TeacherRatingAggregate — ratings

teacher_id FK	→ TeacherProfile
rating_period_id FK	→ RatingPeriod
avg_* (×6) / total_count	float/int
UQ*(teacher,period)	≥5 ⇒ public

Notification — notifications

id PK	bigint
school_id FK	→ School
recipient_id FK	→ User null
channel	sms·push·email·in_app
title / body	char/text
status	queued·sent·failed·read
sent_at / read_at	datetime

Delivery is async via Celery/Redis. Status transitions `queued → sent → read` (or `failed`).

How records are born and move

① ADMISSIONS → ENROLLMENT

Public form
/public/schools/<slug>/apply

→ **AdmissionApplication**
status=pending + track_token

→ **Admin review**
approve / reject →

Student (+User, Guardian)
created_student FK back-links

② BIOMETRIC → ATTENDANCE

Device punch
POST /api/devices/punch/ (token)

→ **BiometricPunch**
raw stored · status=unresolved

→ **Enrollment lookup**
device_user_id → Student/Teacher

→ **Student/StaffAttendance**
present/late by late_after

③ FEES → INVOICE → PAYMENT

FeeStructure + Discounts
amount per year/class

→ **Invoice**
line_items jsonb · grand_total

→ **Payment**
cash/online/bank →

status recompute
paid / partial / overdue

④ EXAMS → RESULTS → PROMOTION

ExamSession + ExamSubject
papers per class

→ **ExamResult**
marks · grade · GPA

→ **is_final ⇒ compute rank**
percentage aggregate →

StudentPromotion
from_class → to_class

CROSS-CUTTING WRITE PATH (EVERY DOMAIN)

Next.js api.js (Bearer + X-Active-School) → nginx /api → DRF ViewSet → TenantMiddleware sets request.school → Serializer validate → SchoolScopedManager (school injected) → Postgres

Endpoints grouped by scope

All under `/api/`. Auth & public routes are open; tenant resources are Bearer-authed and auto-filtered; platform routes require Super Admin.

AUTH & SELF

```
POST /auth/login/
POST /auth/refresh/
GET  /auth/me/
GET  /school/
GET  /device-settings/
```

PUBLIC (NO AUTH)

```
GET  /public/schools/
GET  /public/schools/<slug>/
POST /public/schools/<slug>/apply/
POST /devices/punch/ (token)
```

PLATFORM (SUPER ADMIN)

```
GET  /platform/stats/
/platform/schools/
/platform/leads/
/platform/plans/
```

TENANT RESOURCES — VIEWSETS (1/2)

```
/academic-years/ · /classes/
/subjects/ · /sections/
/routine-slots/
/designations/ · /teachers/
/students/ · /guardians/
/student-transfers/
/certificates/ · /admissions/
/fee-types/ · /fee-structures/
/fee-discounts/
/invoices/ · /payments/
```

TENANT RESOURCES — VIEWSETS (2/2)

```
/exam-sessions/ · /exam-subjects/
/exam-results/ · /promotions/
/student-attendance/
/staff-attendance/ · /leaves/
/biometric-enrollments/
/biometric-punches/
/rooms/ · /room-memberships/
/room-notes/ · /room-posts/
/assignments/ · /submissions/
/rating-periods/ · /teacher-ratings/
/rating-aggregates/ · /notifications/
```

Standard CRUD verbs on each ViewSet: `GET` list/retrieve · `POST` create · `PUT/PATCH` update · `DELETE` destroy. Every queryset is transparently narrowed to `request.school` by the scoped manager — no endpoint can leak cross-tenant rows.